

Doubly Linked List – Insert At End –Space Complexity and Algorithm.

Program :

```
void insertAtPosition(int data, int pos)
{
    if (pos < 1)
    {
        cout << "Invalid position!" << endl;
        return;
    }
    if (pos == 1)
    {
        insertAtBeginning(data);
        return;
    }

    Node *temp = head;
    for (int i = 1; i < pos - 1 && temp != nullptr; i++)
    {
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }

    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;

    newNode->next = temp->next;
    newNode->prev = temp;
    temp->next = newNode;
```

```

    if (newNode->next != nullptr)
    {
        newNode->next->prev = newNode;
    }
    cout << data << " inserted at position " << pos << "."
<< endl;
}

... .

    case 3:
        cout << "Enter data to insert: ";
        cin >> data;
        cout << "Enter position: ";
        cin >> pos;
        insertAtPosition(data, pos);
        break;

```

Space Complexity

Auxiliary Space

Auxiliary space is the extra memory used by an algorithm, excluding the space taken by the inputs.

In the `insertAtPosition` function, the new memory usage is:

- `Node *newNode`: A single new node is allocated. The size of this node is **constant**.
- `Node *temp`: A temporary pointer is created to traverse the list. The size of a pointer is also **constant**.
- `int i`: An integer variable for the `for` loop counter, which occupies a **constant** amount of space.

Since the amount of extra memory does not grow with the size of the linked list, the auxiliary space complexity is $O(1)$.

- **Auxiliary Space: $O(1)$.**

Input Space

Input space is the memory used to store the function's inputs. This can be viewed in two ways.

1. Parameter-only Definition

This perspective considers only the space taken by the function's parameters: `void insertAtPosition(int data, int pos)`.

- The inputs are two integers, `data` and `pos`, each of which occupies a fixed amount of memory.

Under this common definition, the input space is constant.

- **Input Space (Parameter-only): $O(1)$.**

2. Full-Data Definition

This broader view includes the space taken by the entire data structure the function operates on—in this case, the doubly linked list.

- If the linked list contains ***n*** nodes, the total memory it occupies is proportional to ***n***.

Under this definition, the input space is linear with respect to the size of the list.

- **Input Space (Full-Data): $O(n)$.**

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In typical analysis, the "space complexity" of a function refers to its auxiliary space. Therefore, the most common and expected answer for this function is $O(1)$. The for loop traversal affects the function's time complexity ($O(n)$ in the worst case), but not its space complexity.

Algorithm: Insert at a Given Position (Doubly Linked List)

DINSPOS(START, ITEM, POS)

*((Node *)malloc(sizeof(Node)) is availability of space for insertion of Data,
hence, let's name it AVAIL.*)

1. [CHECK FOR INVALID POSITION]

If POS < 1, then:

Write: "Invalid position!" and EXIT.

[End of If structure.]

2. [CHECK FOR INSERTION AT BEGINNING]

If POS = 1, then:

Call DINSATBEG(START, ITEM) [Use the beginning insertion algorithm].

Translating DINSBEG(START, ITEM):

DINSATBEG(START, ITEM)

*((Node *)malloc(sizeof(Node)) is availability of space for insertion of Data,
hence, let's name it AVAIL.*)

1. [ALLOCATE NEW NODE]

Set NEW := AVAIL.

2. [MEMORY ALLOCATED?]

If NEW = NULL, then:

Write: "Error: Memory allocation failed." and EXIT.

[End of If structure.]

3. [COPY DATA]

Set INFO[NEW] := ITEM.

4. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

a. Set PREVLINK[NEW] := NULL.

b. Set NEXTLINK[NEW] := NULL.

c. Set START := NEW.

d. Write: ITEM, "inserted at the beginning."

ELSE: [List is not empty]

a. Set PREVLINK[NEW] := NULL.

b. Set NEXTLINK[NEW] := START.

c. Set PREV[START] := NEW [Point the original first node's PREV back to the new node].

d. Set START := NEW.

e. Write: ITEM, "inserted at the beginning."

[END OF IF STRUCTURE]

5. *Exit*

EXIT.

[End of If structure.]

3. [TRAVERSE TO THE NODE BEFORE THE TARGET POSITION]

Set PTR := START [PTR is a pointer initialized to the start of the list].

FOR(*SET COUNT := 1 ; COUNT < (POS - 1) AND PTR ≠ NULL ; SET COUNT := COUNT + 1*)**DO** :

Set PTR := NEXTLINK[PTR] [Updates PTR to the next node].

[END of Loop]

4. [CHECK IF POSITION IS OUT OF BOUNDS]

If PTR = NULL, then:

Write: "Position out of bounds!" and EXIT.

[End of If structure.]

5. [ALLOCATE NEW NODE]

Set NEW := AVAIL.

6. [MEMORY ALLOCATED?]

If NEW = NULL, then:

Write: "Error: Memory allocation failed." and EXIT.

[End of If structure.]

7. [COPY DATA INTO NEW NODE]

Set INFO[NEW] := ITEM.

8. [LINK THE NEW NODE INTO THE LIST]

a. Set NEXTLINK[NEW] := NEXTLINK[PTR] [New node's NEXT points to what was after PTR].

b. Set PREVLINK[NEW] := PTR [New node's PREV points back to PTR].

c. Set NEXTLINK[PTR] := NEW [PTR now points forward to the NEW node].

d. If NEXTLINK[NEW] ≠ NULL, then: [This handles cases where the node is not inserted at the very end].

Set PREVLINK[NEXTLINK[NEW]] := NEW $\left(\begin{array}{l} \text{The node that was originally after PTR} \\ \text{must now point its PREVLINK back to the NEW node} \end{array} \right)$.

[End of If structure]

9. [OUTPUT THE INSERTED ITEM AT POSITION]

Write: ITEM, "inserted at", POS.

10. Exit
